TECHNICAL REPORT

Predicting the Working Life of Algorithmically Discovered Trading Strategies

A survival-analysis meta-model, built and verified against synthetic ground truth

Author	Robert Stevenson
Repository	strategy-survival-model
Run	seed 7, 5,000 strategies, 5 temporal folds
Source of figures	reports/metrics.json, regenerated by `python scripts/run_pipeline.py`

1. Summary

An automated strategy search produces a queue of candidates that all clear validation. They stop working at very different rates, and the rate determines how much capital a new strategy should get and when it should be reviewed. This report covers a model that predicts how long a newly discovered strategy will keep working, using only the metadata recorded on the day it is deployed.

The finding that matters is not the model's accuracy. It is that the metric allocators implicitly rank on, validation Sharpe, is worse than useless for this question. Ranking strategies by validation Sharpe scores 0.410 on a concordance index where 0.500 is a coin flip, and it stays below 0.500 in all 5 folds. The cause is selection: every strategy entered the queue by clearing a Sharpe threshold, so past that threshold a high score reflects overfitting more often than edge, and overfit strategies decay fastest.

A model reading walk-forward consistency instead reaches 0.781 (95% bootstrap interval 0.772 to 0.790, pooled over 3,000 out-of-fold strategies). Because the data is synthetic, the best score any model could achieve is computable: 0.820. The model sits 0.038 below that ceiling, so most of the remaining error is irreducible noise rather than model capacity.

> Status: the methodology is built and verified against known ground truth. It has not been run on production metadata, because the live book has only been paper trading for a few months and does not yet contain enough resolved strategy lifetimes to support temporal cross-validation.

2. The problem

Strategies discovered by an automated search decay. An edge that clears validation today is usually unprofitable within months, and lifetimes vary by an order of magnitude among strategies with identical headline metrics.

Three operational decisions depend on the decay rate: how much capital a new strategy receives on day one, when its first serious review is scheduled, and when it is retired. The default is to treat every new strategy the same, which misallocates in both directions. It overfunds strategies that will not survive the quarter and underfunds the ones that would have run for a year.

The question is whether discovery-time metadata carries enough signal to separate them. Strategies die for causes that leave traces in that metadata. A strategy selected from a hundred thousand candidates carries more selection bias than one selected from a hundred. A strategy whose walk-forward Sharpe was already sliding during validation is saying something. None of that is visible in a single validation statistic, and all of it is already recorded.

3. Why ranking by validation Sharpe fails

Every strategy in the population cleared a validation-Sharpe threshold of 0.8, because that is how a strategy enters a deployment queue. An observed Sharpe is the sum of true edge, an overfitting component, and measurement noise. Conditioning on that sum exceeding a threshold means the survivors with the highest observed scores are disproportionately the ones whose overfitting and noise happened to break upward.

Past the threshold, additional observed Sharpe is more likely inflation than edge. Because the inflated strategies are the overfit ones, and overfit strategies decay fastest, higher validation Sharpe actively predicts *shorter* working life. The result is not a weak predictor but an inverted one: 0.410 pooled, ranging from 0.354 to 0.427 across folds, never once above the 0.500 of a coin flip.

This is what makes the modeling problem worth posing. Had validation Sharpe ranked survival correctly there would be nothing to add. A meta-model earns its place precisely because the selection process has already consumed the obvious signal.

4. Data

All results come from synthetic data. The generator reproduces the statistical structure of a strategy-search pipeline, including selection bias, walk-forward statistics, regime exposures and censored lifetimes, without

containing anything proprietary.

Confidentiality is one reason. Verification is the more important one. Because the generating process is known, two checks become available that real data cannot support. The best achievable score can be computed exactly, which bounds any claim about model quality. And the model's feature attributions can be compared against the mechanisms actually built in, which turns interpretation into a test with a right answer. The test suite enforces the first of these: one test asserts the model never outscores the oracle, since beating perfect information would indicate a leak.

The run reported here draws 5,000 strategies with seed 7, discovered between 2021-07-01 and 2026-06-01, observed to 2026-07-01. Median observed lifetime is 91 days. 6.8% of strategies are right-censored, either still running at the observation cutoff or administratively retired at a rate of 6%. Survival time is log-normal in the latent quantities with a scale of 0.55 on log-days, which is the noise that places the ceiling below a perfect score.

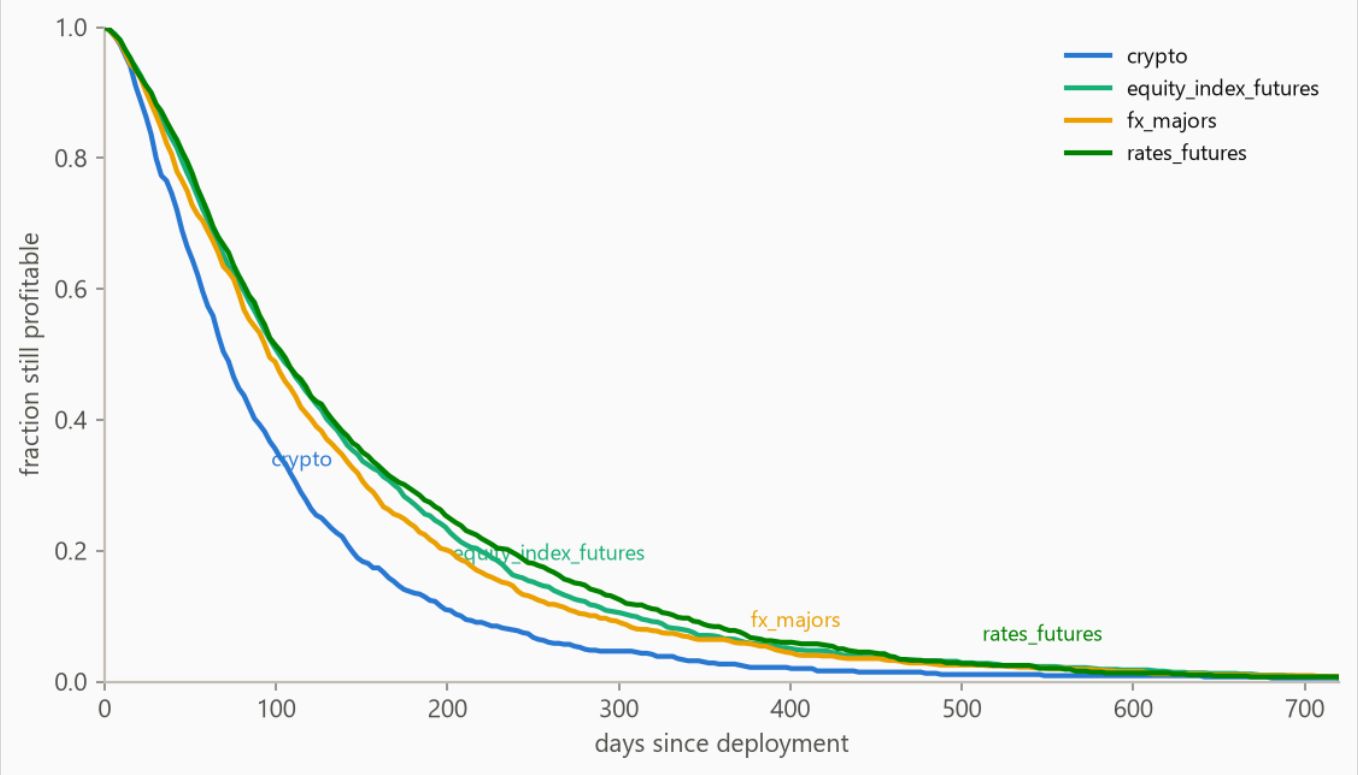

Figure 1. Kaplan-Meier survival curves by asset class. Crypto strategies decay faster by construction, and the model has to recover that from roughly 15% of rows.

5. Method

5.1 Model class

The target is a duration with incomplete observations, so the model is an accelerated failure time (AFT) model: XGBoost with the `survival:aft` objective. Censoring enters through interval labels, where an observed death is the interval [t, t] and a censored strategy is [t, infinity).

Regression on lifetime was rejected because censored rows have no target, and dropping them discards the longest-lived strategies, which biases the model toward pessimism. Classification at a fixed horizon handles censoring awkwardly and answers only one question. AFT keeps every row and returns a full time distribution, which collapses to any horizon an allocator asks about.

5.2 Temporal validation and label re-censoring

Evaluation uses 5 expanding-window folds ordered by discovery date. The earliest 40% of strategies is burn-in that is only ever trained on. Each fold trains on every strategy discovered before its split date and tests on the next 600 strategies, so training sets grow from 1,999 to 4,399 rows.

Training labels are re-censored at each split date. A strategy discovered two years before a split may have died three months after it, and its recorded label contains that future death. Any model trained at that split date could only have known the strategy was still running, so every post-split death is rewritten as a censoring at the split. Omitting this step raises measured scores while silently importing future information, which makes it the most consequential detail in the pipeline. It is implemented as a standalone function (`cv.recensor`) with dedicated tests, because it transfers unchanged to any duration problem on operational data.

Test labels use the full observation window. The model may not see the future; the scorer may.

5.3 Hyperparameter selection and calibration

Hyperparameters are selected once, on the first fold's training window, using an inner temporal split and held-out censored log-likelihood. Likelihood rather than concordance drives selection, and the reason is a failure this pipeline recorded rather than avoided.

An earlier version selected on concordance and looked correct: ranking metrics landed where they ultimately did. It then lost to a no-skill reference forecast on 365-day Brier score. A concordance index is invariant to the predictive scale, so the search had no reason to prefer a usable distribution width and settled on one far too wide. Nothing in the training loop objected, because nothing in the training loop was measuring it. Selection now uses likelihood, which penalizes a broken scale, and the predictive log-normal scale is fitted separately on

a temporal tail slice the early-stopping probe never trained on. The selected loss scale was 0.6, and the calibrated predictive scale came out at 0.64.

The general lesson holds beyond this project. A model can rank well and misstate probabilities at the same time, and the only defense is evaluating both.

5.4 Baselines

Three references bound the result. A Cox proportional hazards model on the same features is the standard survival alternative and answers whether the gradient-boosted model was necessary. Ranking by validation Sharpe is the heuristic a backtest-driven allocator applies implicitly. The oracle ranks by the latent quantity that generated the lifetimes and gives the ceiling that measurement noise imposes.

6. Results

6.1 Discrimination

Pooled over 3,000 out-of-fold strategies, with 95% percentile bootstrap intervals over 500 resamples:

Table 1. Concordance index by method, pooled across folds. Higher is better; 0.500 is a coin flip.

METHOD	HARRELL C	95% INTERVAL
Oracle on latent log-time (ceiling)	0.820	not resampled
XGBoost AFT (pooled)	**0.781**	**0.772 to 0.790**
XGBoost AFT (fold mean)	0.781	not resampled
Cox proportional hazards (fold mean)	0.781	not resampled
Rank by validation Sharpe	0.410	0.397 to 0.422

Two results in that table need stating plainly rather than being left to inference.

The boosted model ties the Cox baseline, 0.781 against 0.781. Both figures are fold means, which is the only like-for-like basis: each fold refits Cox, and its risk scores are relative ones centred on that fold's own training window, so they carry no common scale across folds and pooling them would compare different units. The generator's observable structure is close to additive, and 1,999 to 4,399 training rows is not enough for trees to find much beyond what a penalized linear model in the log-hazard already captures. I report the tie rather than adding interactions to the generator until the headline model wins, because a benchmark tuned until it

loses is not a benchmark. The tie is itself informative: on this problem a simpler model suffices, and knowing that is worth more than an engineered margin.

Both models sit about 0.04 below the oracle. The gap between 0.781 and a perfect score is therefore mostly unpredictable variation in when a strategy actually stops working, not unused signal.

Table 2. Per-fold results. Censoring is the share of each fold's test strategies still running at the observation cutoff.

FOLD	SPLIT DATE	TRAIN N	TEST N	CENSORED	AFT	COX	SHARPE	ORACLE
1	2023-06-22	1,999	600	2%	0.780	0.783	0.425	0.824
2	2024-02-03	2,600	600	2%	0.769	0.771	0.421	0.808
3	2024-09-01	3,199	600	2%	0.768	0.765	0.413	0.805
4	2025-04-03	3,800	600	9%	0.773	0.769	0.427	0.811
5	2025-11-03	4,399	600	39%	0.816	0.816	0.354	0.846

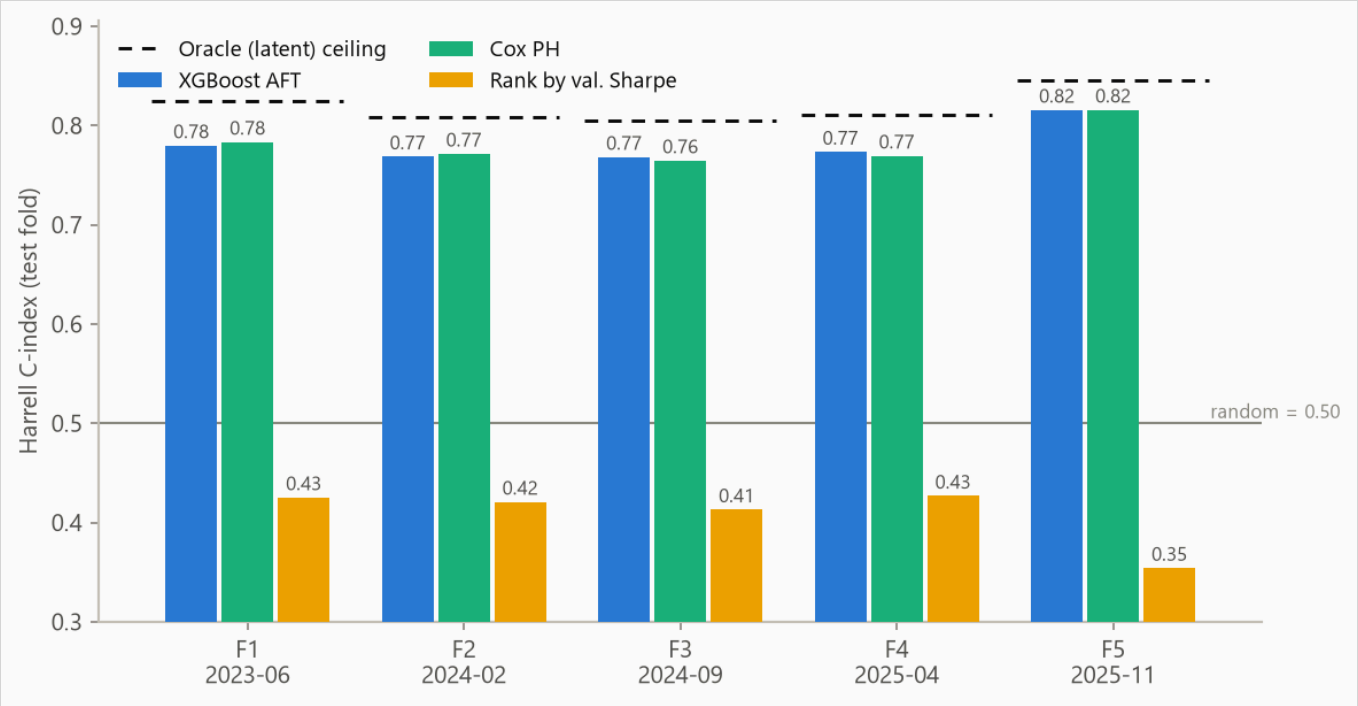

Figure 2. Concordance by fold. AFT performance is stable from 0.768 to 0.816. The final fold scores highest, but 39% of its test strategies are censored against 2% in the first fold, which changes the set of comparable pairs. Concordance is not strictly comparable across folds with different censoring mixes, so this plot supports a claim of stability rather than improvement.

6.2 Calibration

Predicted median survival times are converted to survival probabilities under the calibrated log-normal, then checked two ways. Brier scores are weighted by the inverse probability of censoring so that censored rows do not bias the result. The no-skill reference assigns every strategy the pooled Kaplan-Meier marginal, ignoring all features.

Table 3. IPCW Brier score by horizon. Lower is better.

HORIZON	AFT	COX	NO-SKILL MARGINAL
90 days	0.148	0.150	0.249
180 days	0.132	0.132	0.182
365 days	0.050	0.051	0.056

The margin over the no-skill reference is wide at 90 days and narrow at 365. That is the expected shape. Median survival is about 91 days, so by one year nearly every strategy has stopped working and the marginal forecast is already close to certain. Little skill remains to demonstrate at that horizon.

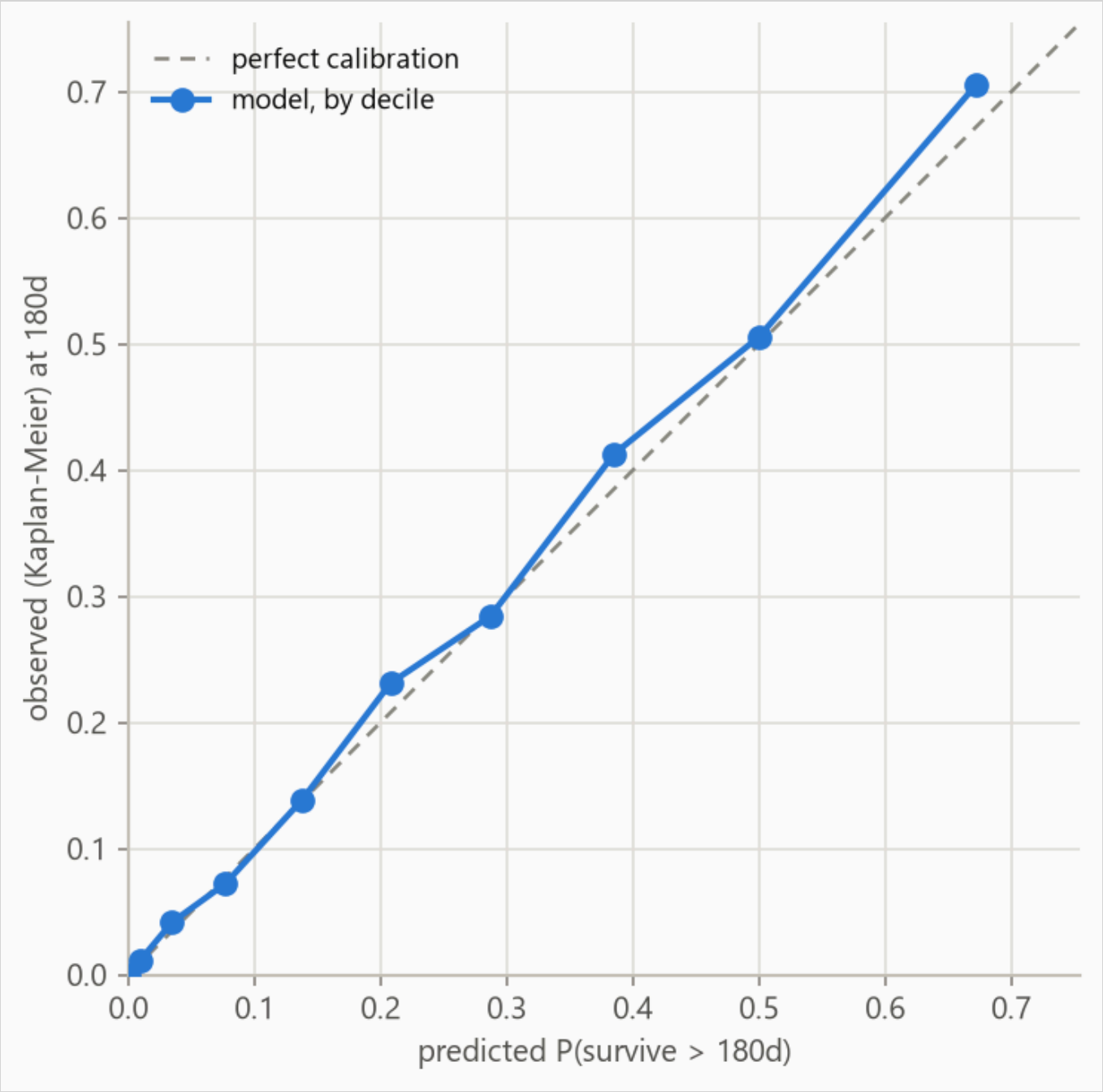

Figure 3. Predicted against observed survival at 180 days, by predicted decile. Observed frequencies are Kaplan-Meier estimates within each bin, so censored strategies contribute correctly rather than being dropped. The largest deviation is 0.033 in decile 10.

Table 4. Decile calibration at 180 days, the values plotted in Figure 3.

DECILE	N	PREDICTED	OBSERVED (KM)	DEVIATION
1	300	0.001	0.000	-0.001
2	300	0.010	0.012	+0.002
3	300	0.035	0.042	+0.007
4	300	0.077	0.072	-0.005
5	300	0.138	0.139	+0.001
6	300	0.208	0.231	+0.023
7	300	0.287	0.284	-0.003
8	300	0.385	0.413	+0.028
9	300	0.500	0.506	+0.006
10	300	0.672	0.705	+0.033

7. What the model uses

Feature attributions are computed on the log-time margin, so a value of +0.3 multiplies predicted survival time by about 1.35 and negative values shorten it.

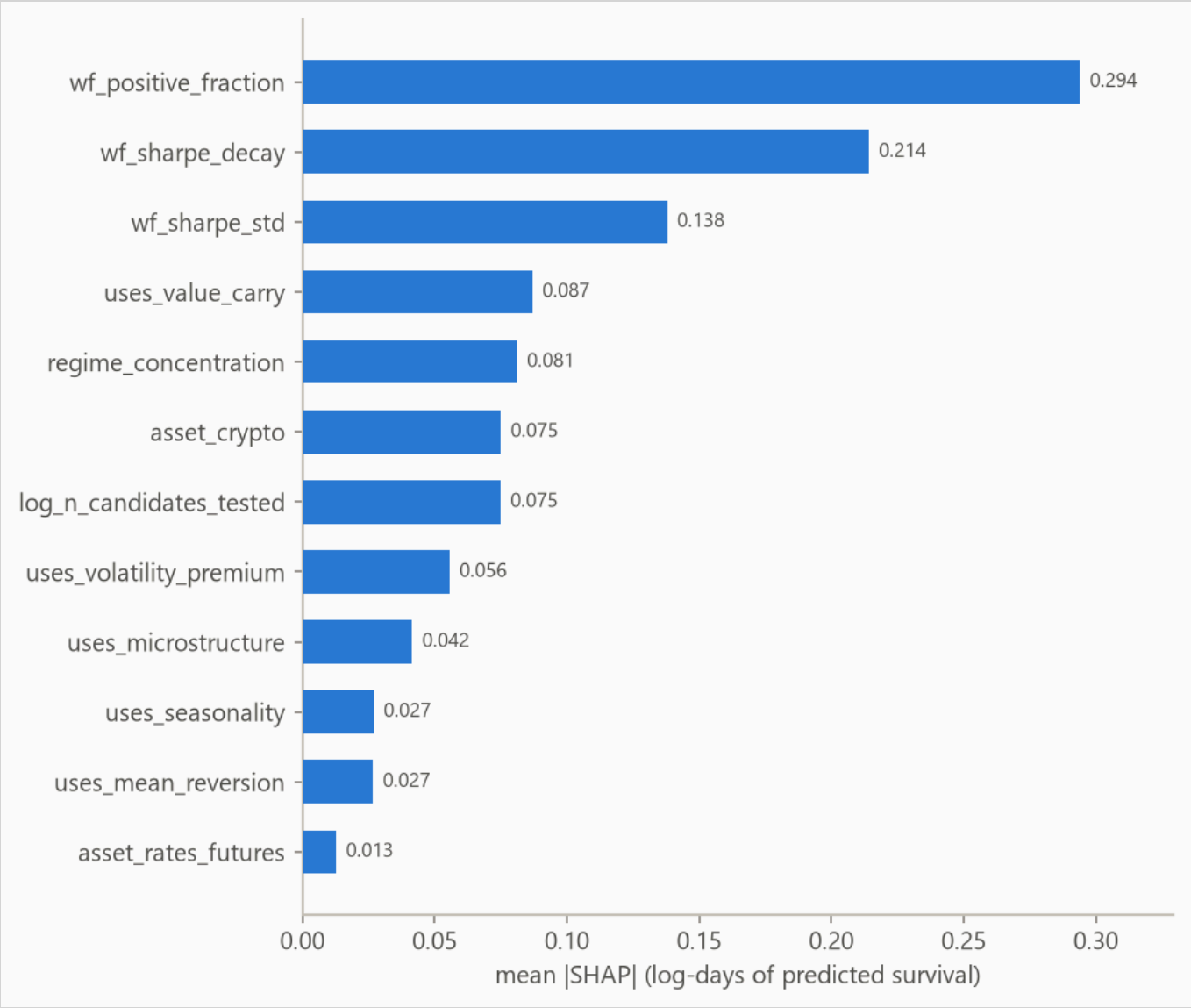

Figure 4. Mean absolute attribution by feature. The three walk-forward consistency statistics carry more attribution than every other feature combined.

Table 5. Top eight features by mean absolute attribution.

FEATURE	MEAN |ATTRIBUTION|
wf_positive_fraction	0.294
wf_sharpe_decay	0.214
wf_sharpe_std	0.138
uses_value_carry	0.087
regime_concentration	0.081
asset_crypto	0.075
log_n_candidates_tested	0.075
uses_volatility_premium	0.056

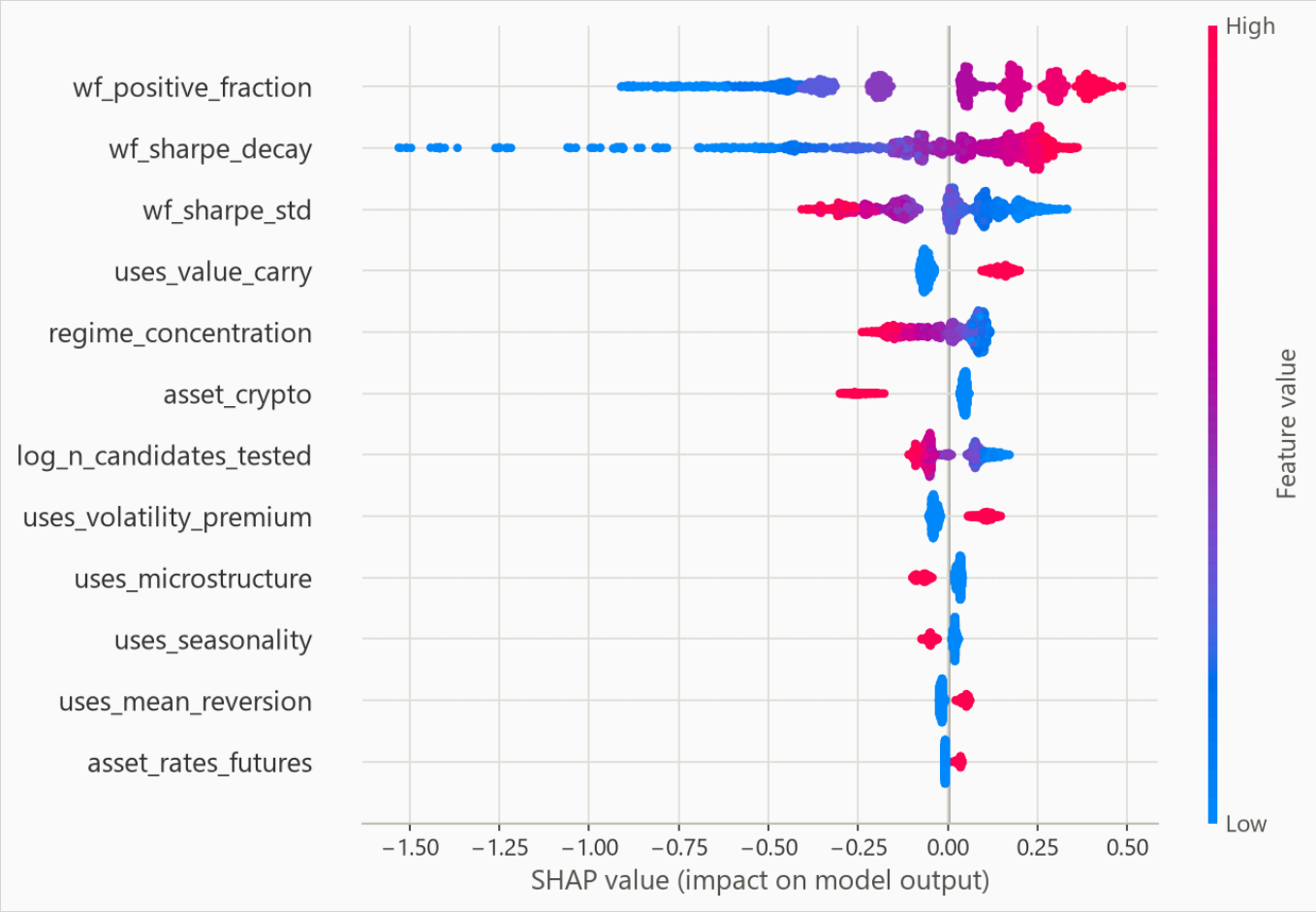

Figure 5. Per-strategy attributions. High walk-forward positive fraction pushes predicted survival up; a steeply negative walk-forward Sharpe slope pushes it down hard, with the left tail reaching about -1.5 log-days, a factor of roughly 4.5 shorter; high fold-to-fold Sharpe dispersion is penalized. All three directions match the generative design, where consistency rises with true edge and falls with overfitting.

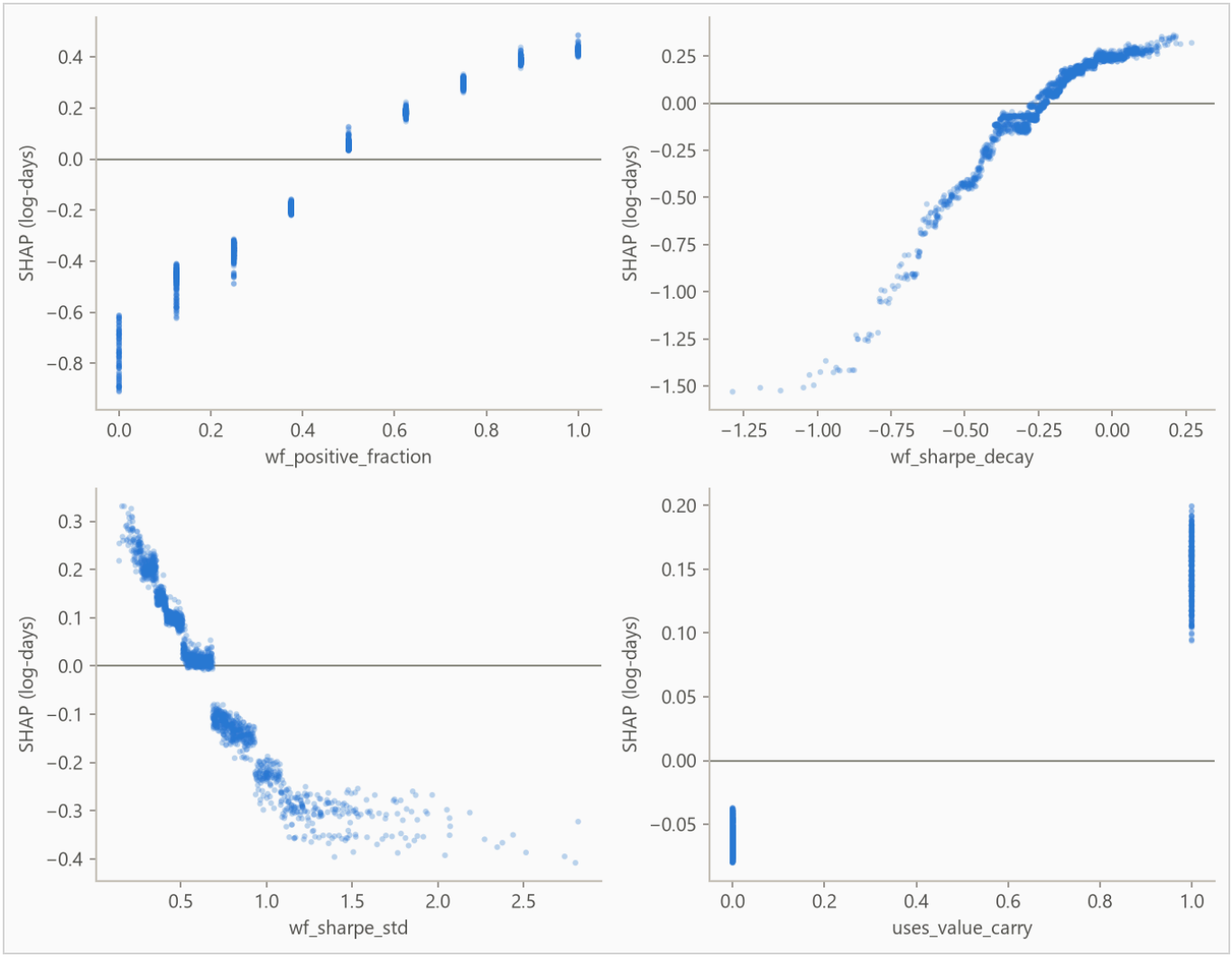

Figure 6. Attribution against feature value. The positive-fraction effect is monotone and close to linear, with vertical bands at the nine possible fold fractions and a swing of about 1.3 log-days end to end. The Sharpe-decay effect saturates once the slope turns positive, which is sensible behaviour given that positive decay slopes are mostly noise. The dispersion effect is a descending staircase that flattens past 1.5, where additional dispersion no longer distinguishes anything.

Validation Sharpe does not appear in the top twelve features. On unconditional data that would be strange. Here it is the signature of selection: every strategy cleared the same threshold, so the metric's remaining variation is mostly overfitting plus noise, and the model routes its attention to walk-forward consistency instead. This is the same phenomenon as the 0.410 in Table 1, seen from inside the model.

The attribution ranking is a test rather than a description, because the generating process is known. The three walk-forward statistics that dominate are not inputs to survival time in the generator. They exist only as the least noisy observable proxies for the latent split between real edge and overfitting, which is the actual driver. The model was not handed that relationship; it found the proxies and leaned on them. Feature-family and

asset-class effects return with the signs and rough ordering written into the generator, and the search-intensity penalty appears where a multiple-testing argument predicts.

Directions in that table are more trustworthy than magnitudes. The three walk-forward statistics are correlated proxies for the same latent quantity, and how attribution divides credit among correlated features reflects the model's internal choices as much as the data. The supportable reading is that the model uses positive fraction about twice as much as Sharpe dispersion, not that positive fraction is twice as important. The feature-family and asset-class effects enter the generator additively and independently, so any reasonable model recovers them; recovering the proxy structure is the informative result.

8. Limitations

The generator encodes a prior, not evidence. A model that recovers structure placed there says nothing about whether real strategy metadata contains comparable signal. This work validates a methodology, and the concordance reported here should be treated as an upper bound on production performance.

Lifetimes are treated as independent. Real strategies stop working together when a regime breaks. The generator omits that dependence and the bootstrap resamples strategies independently, so the intervals in Table 1 understate real uncertainty and are best read as a lower bound on it.

Administrative retirement is modeled as independent censoring. In a live book, strategies are retired because someone observed early decay, which makes the censoring informative and biases survival estimates upward. Handling it properly requires a competing-risks treatment, which is recorded as future work rather than implemented.

Generator-seed dependence. The pipeline was rerun end to end with a second seed (seed 8, saved as `reports/metrics_seed8.json`). It selected the same hyperparameters and scored 0.781, inside the seed-7 interval of 0.772 to 0.790. Ranking by validation Sharpe stayed inverted at 0.398, and the oracle ceiling came out at 0.812. The same three walk-forward statistics dominate attribution, in the same order. Two seeds is a consistency check rather than a variance estimate. A proper multi-seed sweep remains future work, and the relationships between the numbers in this report deserve more confidence than their individual decimals.

Harrell's concordance is biased under heavy censoring. The final fold is 39% censored, where an inverse-probability weighted concordance would be preferable. Fold-to-fold comparisons in Table 2 should be read with that in mind.

9. Intended use

The model is a capital-allocation and review-scheduling prior, not a trade signal. It ranks newly discovered strategies by expected working life and produces a survival curve for each. Predicted median lifetime sets initial position sizing and the first review date; the curve gives a decay schedule against which actual performance can be compared.

Before it informs live allocation, the same temporal cross-validation with the same label re-censoring has to be repeated on production metadata, and the resulting concordance is expected to be lower with wider intervals. The blocking constraint is data volume rather than method. The live system has been paper trading for a few months, which is not enough resolved lifetimes to support 5 temporal folds with meaningful censoring. The pipeline is built and verified against known ground truth so that when the book is old enough, the production run needs no methodological work.

The real-data path itself already exists rather than being planned: the same pipeline fits and evaluates any right-censored duration CSV (`scripts/run_fit_evaluate.py`), saves both fitted models, and scores new rows later (`scripts/run_predict.py`). The repository includes a demonstration on 262,763 real City of Chicago business licences (`reports/chicago_demo/`), where the checks that depend on ground truth are absent and the generated report says so.

Nearer-term work, in the order it would be done: a multi-seed sweep to separate data variance from model variance, an inverse-probability weighted concordance alongside Harrell's, a block bootstrap by discovery month to stop understating interval width, and a competing-risks treatment of administrative retirement.

10. Reproducing these results

Python 3.11 or later. From the repository root:

```
pip install -r requirements.txt
python -m pytest
python scripts/run_pipeline.py
```

The test suite is 77 tests covering generator invariants, fold and label leakage, and metric behaviour. The pipeline script regenerates the data at seed 7, runs the 5-fold temporal cross-validation, writes `reports/metrics.json` and all figures in about two minutes, then rebuilds this report from that metrics file, so every number here is traceable to a single run rather than transcribed. `requirements.txt` pins the exact dependency versions these numbers were produced with.

The seed-dependence paragraph in section 8 is generated from `reports/metrics_seed8.json`, produced by `python scripts/run_pipeline.py --seed 8` and saved under that name; the builder reads it when present and states the single-seed limitation when it is not. Building the report also prints `strategy_survival_report.pdf` headlessly when Chrome is available.

Generated from `reports/metrics.json` by `scripts/run_build_report.py`. Seed 7, 5,000 strategies, 3,000 out-of-fold test strategies.